

Centro Estadual de Educação Tecnológica Paula Souza

ETEC Profª Maria Cristina Medeiros

Aula 02

Componente Curricular: Sistemas Web II

1. Objetivos

- Consolidar os endpoints **GET** por id e **DELETE** utilizando consultas reais ao banco de dados.
- Implementar o método **PUT** para atualização completa de um registro persistido no MySQL.
- Compreender o papel do *commit* (salvar a alteração no banco) e do *refresh* (atualizar o objeto Python com os dados mais recentes do banco).
- Aplicar, de forma autônoma, a criação de uma nova tabela e endpoints CRUD completos para um novo recurso, utilizando banco de dados real.

2. Conteúdo programático

- Correção e discussão da atividade prática da aula anterior (**GET** por *id* e **DELETE** com banco real).
- Método **PUT**: atualização completa de um registro no banco de dados.
- **Ciclo de uma escrita no banco**: alterar o objeto, dar commit, e opcionalmente dar refresh.
- Consolidação do CRUD completo (Create, Read, Update, Delete) com persistência real.

3. Desenvolvimento

3.1 Abertura

Retomar rapidamente a configuração da conexão com o MySQL feita na aula anterior (`database.py`, `models.py`, `schemas.py`) e os endpoints GET `/produtos` e POST `/produtos` já implementados com persistência real.

3.2 Correção guiada da atividade prática

```
# main.py (trecho adicionado)
from fastapi import HTTPException

# GET /produtos/{id} -> consulta um produto pelo id no banco
@app.get('/produtos/{produto_id}', response_model=ProdutoResponse)
def obter_produto(produto_id: int, db: Session = Depends(get_db)):
    produto = db.query(ProdutoDB).filter(ProdutoDB.id ==
produto_id).first()
```

```

        if produto is None:
            raise HTTPException(status_code=404, detail='Produto não
encontrado')
        return produto

# DELETE /produtos/{id} -> remove um produto do banco
@app.delete('/produtos/{produto_id}', status_code=204)
def remover_produto(produto_id: int, db: Session = Depends(get_db)):
    produto = db.query(ProdutoDB).filter(ProdutoDB.id ==
produto_id).first()
    if produto is None:
        raise HTTPException(status_code=404, detail='Produto não
encontrado')
    db.delete(produto)
    db.commit()

```

3.3 Método PUT

Implementar a atualização completa de um produto já existente:

```

# PUT /produtos/{id} -> atualiza um produto existente no banco
@app.put('/produtos/{produto_id}', response_model=ProdutoResponse)
def atualizar_produto(produto_id: int, dados: ProdutoCreate, db:
Session = Depends(get_db)):
    produto = db.query(ProdutoDB).filter(ProdutoDB.id ==
produto_id).first()
    if produto is None:
        raise HTTPException(status_code=404, detail='Produto não
encontrado')

    produto.nome = dados.nome
    produto.preco = dados.preco
    produto.quantidade = dados.quantidade
    db.commit()
    db.refresh(produto)
    return produto

```

Ciclo de uma escrita no SQLAlchemy:

- os atributos do objeto produto são alterados em memória; `db.commit()` efetivamente grava essas alterações no MySQL;
- `db.refresh(produto)` atualiza o objeto Python com o estado mais recente do banco (útil quando o banco gera ou recalcula algum valor).

3.4 Testando o CRUD completo

Testar, pela documentação interativa do FastAPI (/docs), o fluxo completo do CRUD com persistência real: cadastrar (POST), listar (GET), consultar por id (GET/{id}), atualizar (PUT) e remover (DELETE) — e, opcionalmente, consultar diretamente a tabela pelo MySQL Workbench para confirmar que os dados realmente mudaram no banco.

3.5 Prática

Cada aluno deve criar, em um novo conjunto de arquivos (ou adicionando ao projeto existente), uma API REST com banco de dados real para um recurso diferente de Produto (por exemplo: *Aluno*, *Livro*, *Cliente* ou *Tarefa*), definindo:

- Um modelo ORM (SQLAlchemy) com pelo menos 4 atributos, mapeado para uma nova tabela.
- Um schema Pydantic de entrada e de saída.
- Os cinco endpoints do CRUD (GET, GET por id, POST, PUT, DELETE), todos operando sobre o banco de dados real.

3.6 Entrega da atividade

- Enviar os arquivos para o seu repositório do github e colocar o link deste projeto na atividade disponibilizada no Teams.
- Próxima aula tratará da escrita de testes unitários — e que, agora que os endpoints dependem de um banco de dados real, será necessário aprender a simular essa dependência durante os testes.

4. Referências

- Documentação oficial do SQLAlchemy — Session Basics:
https://docs.sqlalchemy.org/en/20/orm/session_basics.html
- Material e exemplos de código disponibilizados em aula.